ChatGPT

Proposed Architecture Overview

Designing a robust backend architecture for the application requires choosing stable technologies and ensuring data persistence, especially given offline usage scenarios. We will limit the stack to **TypeScript-based** backend services (for type safety and modern language features) and use **Docker** containers for deployment consistency. A **RESTful API** will serve as the interface for clients (e.g. the React front-end or mobile app), providing endpoints to create, read, update, and delete data (e.g. fumigation jobs, field records, user info). Key considerations include selecting a suitable web framework, deciding between a relational or non-relational database, and handling "ephemeral" jobs or offline-generated data by persisting state externally so nothing is lost on service restarts. The architecture should be modular, scalable, and aligned with best practices for stability.

REST API with TypeScript (Framework Selection)

For the TypeScript backend, we need a **stable and well-supported web framework** to build the REST API. While **Express.js** is a popular minimalist choice, offering flexibility and a huge ecosystem, it provides only the basics out-of-the-box [1]. This simplicity is great for quick development, but larger projects can become unwieldy without a defined structure. A more **opinionated framework like NestJS** is often recommended for enterprise-scale TypeScript projects [1]. NestJS is built with TypeScript in mind and enforces a modular architecture (inspired by Angular's design), which encourages best practices (e.g. controller-service-repository patterns, dependency injection) and leads to more maintainable code in the long run [1]. In other words, **Express** gives maximum freedom and is very mature, but **NestJS** provides "a full toolbox" for scalability and maintainability, making it *"ideal for large-scale applications"* [1].

Given the requirement for *"the best and most stable"* solution, NestJS would be a strong candidate due to its structured approach and active community. It sits on top of either Express (default) or Fastify, combining maturity with modern patterns. That said, Express itself is proven and could be used with a manual layered structure if simplicity is preferred [2] [3]. Other frameworks like **Fastify** (for higher performance) or **Hapi** (with a configuration-driven approach) are also options, but Express/Nest are the most widely adopted in the Node/TS ecosystem as of 2025 [2] [4]. In summary, **NestJS** is recommended for its stability and built-in features, whereas **Express** remains a solid choice for a lightweight service – the decision hinges on whether you value a predefined architecture over flexibility [4]. Either way, using TypeScript with these frameworks will ensure type safety across the codebase.

Database Choice (Relational vs Non-Relational)

Choosing the right database is critical for both performance and data integrity. We need to analyze whether a **relational (SQL)** or **non-relational (NoSQL)** database is more suitable for this application's needs. Traditional **SQL relational databases** (like PostgreSQL or MySQL) offer strong ACID guarantees (Atomicity, Consistency, Isolation, Durability) which ensure reliable transactions and consistency of data [5]. They also allow powerful querying with joins and aggregations, making them ideal for structured interconnected data and analytics [6]. These qualities have made relational DBs the default choice for decades due to their *"battle-tested reliability"* and well-understood behavior [7]. In our context, if the data model involves clear relationships (e.g. users, their projects, tasks, etc.) and we need complex queries (such as reporting across many records), a relational database would work well.

On the other hand, **NoSQL databases** offer a more flexible, schema-less design and scale horizontally with ease [7]. A NoSQL (non-relational) store can handle semi-structured or evolving data models without requiring rigid schemas or migrations for every change. In fact, one of the biggest advantages is **schema flexibility** – developers aren't constrained by a fixed table structure and can add new fields or JSON structures on the fly with minimal effort [8]. This can greatly speed up iteration when the data requirements are changing, as there is no need for costly alter-table operations or downtime for migrations [8]. The trade-off is that NoSQL systems often sacrifice some consistency or complex query capability for this flexibility and scalability. Many NoSQL databases follow the BASE model (Basically Available, Soft state, Eventual consistency) rather than full ACID transactions [9]. This means writes and reads are very fast and distributed, but data might only become consistent over a short time rather than instantly [9]. For example, if two devices update a record concurrently offline, a conflict resolution strategy is needed when syncing in a NoSQL system. For an app that can tolerate eventual consistency (e.g. two users' data syncing may not be perfectly in sync at every millisecond, as long as it converges), NoSQL is acceptable and often preferred for performance.

Given our application's nature (field operations, fumigation jobs, etc.), we need to consider how data is used. The app likely handles entities like **Jobs/Tasks**, **Field data (geo-coordinates, etc.)**, and **User profiles**. These can be modeled in a relational way (with foreign keys linking jobs to a user, etc.), but a **document-based NoSQL** might be equally or more convenient: for instance, a "Job" document could embed all relevant info (user ID, field polygon, timestamps, etc.) as a JSON object. If we anticipate a need for **offline data storage and sync** (which seems likely given the prior offline-first approach), a NoSQL document database aligns well. It allows the client to store records locally as JSON and push them to the server DB when online, without having to translate to normalized tables. Indeed, modern apps at scale often introduce NoSQL for *"flexible schema design, horizontal scalability"*, allowing fast iteration and distribution [7]. Developers often prefer the closer alignment to in-app objects – *"NoSQL felt closer to the way I worked with objects… less friction"* as one report put it [10].

Recommendation: Use a **NoSQL document database** for this project, due to its flexibility and alignment with the likely usage patterns. A document DB (like **MongoDB**, **CouchDB**, or **Firebase Firestore**) will let us store data in JSON format and easily update the schema as the application evolves [8]. It's also well-suited for cloud deployment and scaling if the user base grows. For instance, if we choose MongoDB, we can containerize it and use Mongoose (in a Node/TS app) for structured ODM modeling. If we choose CouchDB, we gain the benefit of its built-in sync protocol (more on that in the next section). Firestore (Google's cloud NoSQL) is another option if managed service is desired; it natively handles offline caching on clients and sync. Overall, a NoSQL store will simplify storing the "jobs" and related data, as we can directly persist the objects the application uses without complex relational mapping. **However**, we should also be mindful of consistency – implementing application-level checks for critical data integrity since the DB won't enforce relationships as strictly as SQL. For example, ensure a job references a valid user ID, etc., either via code or using features like MongoDB's schema validation. If down the line we find certain data highly relational (say user accounts and payments), we could adopt a hybrid approach (using SQL for those parts), but to start, a single NoSQL database is sufficient and avoids over-complicating the architecture [11].

Handling State: Jobs, Offline Data, and Persistence

One of the challenges mentioned is that *"jobs could be created in the air and when they come back they don't exist"*. This implies that if the application or service is offline or temporarily down, any tasks (jobs) created only in memory might vanish. The architecture must therefore **persist state externally** so that no work is lost due to process restarts or network issues. In cloud-native design, this is addressed by making services **stateless**: the application processes themselves do not retain important state in RAM between requests. Instead, all stateful information (jobs, sessions, progress data) is stored in durable

backing services (databases, caches, etc.) [12] . Following this principle, whenever a new "job" or task is created (e.g. a fumigation job scheduled by a user), the API should immediately write it to the database (or a persistent queue) so that it exists independently of the runtime memory. This way, if the container restarts or the user's connection drops, the job still "exists" in the backend store and can be retrieved later. The **12-Factor App methodology** explicitly notes: *"Each process should be stateless… Persistent data should be stored in a backing service like a database or cache. Processes should be ephemeral — they can be restarted or replaced at any time"* [12] . In practice, this means we will **never only keep a job in memory** after creation – it will always be written to the DB (or a message broker) as a source of truth.

For handling **background or asynchronous jobs** (if any processing needs to happen on these "jobs"), we have a couple of strategies: - Use a **job queue system** (like **Redis** or RabbitMQ) to store and process tasks. For example, when a job is created via the REST API, the service could place a message on a Redis-backed queue and a separate worker process (in another Docker container) would pull from the queue to execute any long-running work. This provides durability (the queue can persist tasks to disk) and decouples job processing from the web API responsiveness. Many Node/TS libraries (like BullMQ for Redis) support this pattern. - If jobs are not computationally heavy and mainly tracking user actions, simply writing their status to the database might suffice. The API can update a job's status (e.g. "pending", "completed") in the NoSQL DB, and clients can poll or subscribe for changes.

The mention of *"like when the progress of games is saved by Google"* hints that users might perform actions offline or on one device, and we want that progress to be available later or on another device. This is essentially an **offline synchronization and state persistence** problem. In the mobile gaming world, Google Play Games solves this by syncing game saves to the cloud under the user's account – when you log into a new device, your progress downloads automatically [13] . We can emulate this approach in our architecture: implement **user authentication** (possibly via Google OAuth or a custom system) and tie all data to the user's account. Then, when a user uses the app on any device and creates or updates data, those changes are sent to the cloud database under their user ID. If they switch devices or reinstall the app, upon login the app can fetch their data from the cloud and "restore progress", just like Google does for games (*"progress will be loaded automatically when you log in… progress will be saved between devices"* [13]).

To support **offline-first usage**, we should allow the client app to cache actions locally and sync when network is available. We have a couple of options: - **Leverage CouchDB/PouchDB replication:** If we choose CouchDB as the server database, the front-end could use PouchDB (a JS library) as a local, in-browser database. PouchDB can transparently sync with CouchDB when online. This architecture has been proven in offline-first apps; essentially, the browser/app writes to a local PouchDB store while offline, and *"once you're back online, sync it"* to the server CouchDB [14] . This would make the app extremely resilient to connectivity issues: the user can create multiple jobs offline, and when a connection is available, the data replicates to the central DB. CouchDB is designed with replication in mind – *"built around the idea of syncing your data"*, enabling the web app to become **offline-first** [15] . The downside is a slightly more complex setup (managing Couch/Pouch and potential data conflicts), but it offers seamless offline capability. - **Alternative approach:** If using a different DB like MongoDB or a cloud service, we can implement a custom sync mechanism. For example, the client app could keep a local log of changes (in IndexedDB or localStorage) and the backend API provides endpoints to batch upload these when online. This is more manual but still effective. Another approach is using a service like **Firebase Firestore**, which has SDKs that cache writes locally and sync in background; however, that would mean relying on Google's infrastructure rather than our own Dockerized service (though Firebase could be an option if we relax the "Docker for all services" requirement).

No matter which approach, the core idea is to **persist every user action or job to a reliable store** as soon as possible. By doing so, we ensure no data is lost. The server will integrate with external services

as needed – for instance, if we allow Google login, we'll integrate with Google's OAuth API (but still store our data in our database). If we use third-party backups (maybe periodically backing up data to cloud storage), that's another integration. But primarily, the database is the system of record.

In summary, to handle ephemeral jobs and offline data: - The **REST API** will be stateless and simply act as a conduit to store/fetch data from the DB. If a user creates a job while online, the API immediately writes it to the DB and returns an ID. If the user is offline, the front-end caches the job and the API will accept it and save to DB once connectivity returns. - **Data is persisted in the NoSQL DB** (or a queue) so that even if the app or server restarts, the data remains. This is analogous to cloud game saves – the data "lives" on the server reliably, not just on the volatile client or memory [13]. - Optionally, use a **distributed cache or queue** (Redis) for transient but critical data (e.g. session tokens or job processing states), with persistence enabled. This can improve performance while still avoiding data loss. - Implement **synchronization logic** for offline support: either using a proven system like CouchDB/PouchDB sync (which *"rather than relying on a master, supports multi-master replication"* allowing each client to sync with the server and others [16]), or a custom approach. The CouchDB route has the appeal of a ready-made solution where *"with PouchDB on the client's browser and CouchDB on the backend, your web app can become offline-first capable"* [14]. This matches the project's original offline-first philosophy. - Ensure **conflict resolution** strategies if multiple updates happen offline – e.g., last write wins or custom merge for specific fields – especially if multiple users can edit the same data. CouchDB provides revision trees for conflicts, which could be useful. If using a simpler approach, the API could reject conflicting edits or use timestamps.

By designing with these principles, we prevent the scenario of "jobs disappearing into thin air." Every job or progress update will be accounted for in a persistent store, much like how a game that saves progress to the cloud allows you to pick up where you left off on any device [13]. In practice, when a field operator comes back from the field (reconnects), the data they entered offline will sync to the server, and later they or others can retrieve it from the central database at any time.

Containerization and Deployment Strategy

Using **Docker for the services** means each component of our system will run in an isolated container, ensuring environment consistency and easy deployment. We will define Docker images for: - **API Service** – a Node.js (TypeScript) application container. This image will contain the code (compiled to JS if needed) and run the web server (Express/Nest) on a specified port. - **Database** – if we self-host MongoDB or CouchDB, this will run in its own container (using an official Mongo/Couch image). Data volumes should be configured so that data can persist between container restarts (or use managed DB in production for reliability). - **Optional Worker Service** – if we implement a separate job worker (for background processing or scheduled tasks), that can be another Node.js container, consuming from a common queue or looking for "pending jobs" in the DB. - **Optional Cache/Queue** – e.g., a Redis container if we use Redis for caching or as a message broker for jobs.

All these services can be orchestrated with **Docker Compose** for local development, allowing us to simulate the whole system (API, DB, etc.) easily. In production, we could deploy the containers to a cloud service. For instance, using **Kubernetes** or a service like AWS ECS or Docker Swarm to manage containers provides scalability: we could run multiple instances of the API container behind a load balancer to handle more traffic, scale out the worker containers if background jobs pile up, etc. Docker ensures that wherever we run the service, it behaves the same (since dependencies and environment are containerized). This contributes to system stability – no "it works on my machine" issues – and allows quick recovery and scaling (spin up new container instances as needed).

Isolation in containers also means we can update or replace components independently. For example, if we later decide to switch the database (say from MongoDB to CouchDB or vice versa), we just replace that service's container and update the API accordingly, without affecting the other containers (adhering to treating backing services as attached resources, per 12-factor principles) [17]. Environment variables (injected at container run time) will be used for configuration (DB connection strings, API keys), so that secrets and configs aren't baked into images, again following best practices [18] [19].

In terms of **networking**, the REST API container will expose ports (e.g. 8080) and likely be fronted by a reverse proxy or load balancer (NGINX or cloud LB) in production. Each container (DB, API, etc.) can communicate within a private Docker network (for security).

Docker also makes local testing easier – we can run the whole stack on a developer's machine or CI pipeline, which is important for continuous integration and testing of the system.

Security & Stability: We should also think about using Docker images that are regularly updated and minimal (to reduce attack surface). For instance, using Node's official Alpine image for a smaller footprint, and perhaps running the Node process as a non-root user inside the container. We will also need to handle data backup for the DB container (mount volumes or use managed storage) to prevent data loss. In production, a managed DB service might be considered for ease, but since the focus is Docker, we assume self-hosting in a container with proper backup strategies (like snapshotting volumes or scheduled dumps).

In summary, containerizing the architecture provides: **portability** (run anywhere), **scalability** (run multiple instances easily), and **isolation** (each service in its own sandbox). The microservices approach can be taken if the app grows – e.g., splitting authentication, jobs processing, and main API into separate services – and Docker will facilitate that by allowing each to be developed and deployed independently. For now, even if it's a single service (monolithic API) plus a DB, Docker adds value by standardizing deployment. The architecture diagram would show the client app interacting with the Node.js API (container) via HTTP, the API reading/writing to the database (container), and any background worker or cache in the mix as well – all these components residing in Docker containers across one or more hosts.

Conclusion

After thorough analysis, the recommended architecture is a **TypeScript-powered REST API** deployed in Docker, backed by a **NoSQL database**, with careful handling of state to support offline scenarios and data durability. The backend API can be built with a framework like NestJS for maximum stability and organization (or Express if we prefer minimalism), exposing endpoints for the client application [1]. Instead of a traditional SQL database, a document-oriented NoSQL DB (e.g. MongoDB or CouchDB) is advised, to allow flexible schemas and easier sync of JSON data between client and server [8]. This database choice aligns with the app's offline-first use case and can scale horizontally as needed [7].

Crucially, the system will be designed so that **no important data "lives" only in volatile memory** – whenever a job or piece of progress is created, it's saved to the persistent store immediately. This means even if the server restarts or the user's device goes offline, the state is not lost [12]. The application will treat backing services (database, etc.) as the source of truth for state, enabling stateless application servers that can be replaced or scaled at will without affecting user data [12]. For offline capability, we can integrate a syncing mechanism: using CouchDB's replication or a custom approach, the user's local data changes will synchronize with the cloud when a connection is available [14]. In

effect, the backend plays a role similar to Google's cloud save – the user's progress/jobs are stored on a central server tied to their account, and can be retrieved on any device upon login [13].

All components are containerized with **Docker**, ensuring consistency across environments and simplifying deployment. This architecture is both **robust** (no single point of failure in memory, data safely in DB) and **scalable** (each service can be replicated in containers). It embraces modern best practices (12-factor app principles for config and state, microservice-ready design) while meeting the project's specific needs like offline usage and future growth. By following this design, the application will be well-equipped to reliably handle user operations (fumigation plans, field data, etc.), even in intermittent network conditions, and provide a stable, maintainable platform for future features.

Sources:

- Ms. Byte Dev, *"Express.js vs. NestJS: Which One Should You Choose in 2025?"*, CodeX (Medium) – comparison of Node.js frameworks [1].
- Leapcell Dev Community, *"The 5 Most Popular Node.js Web Frameworks in 2025"* – notes on Express (simplicity) vs Nest (structure) and others [4].
- Skyvia Blog, Edwin Sanchez, *"SQL vs NoSQL: Choosing the Right Database"* (2025) – discusses rigid schema vs flexible schema trade-offs [8] [9].
- ByteByteGo Newsletter (Jun 2025), *"Relational vs NoSQL Databases"* – highlights strong consistency of SQL vs horizontal scaling of NoSQL [7].
- usePouchDB Documentation, *"Why CouchDB and PouchDB?"* – explains offline-first syncing with PouchDB/CouchDB (multi-master replication) [14].
- Google Play Games Support, *"Transfer my game progress between devices"* – example of cloud-save (progress tied to Google account across devices) [13].
- Dev.to – *"12-Factor App Methodology: Processes"* – importance of stateless processes and externalizing state to backing services [12].

---

[1] Express.js vs. NestJS: Which One Should You Choose in 2025? | by Ms. Byte Dev | CodeX | Medium
https://medium.com/codex/express-js-vs-nestjs-which-one-should-you-choose-in-2025-a63be5908d0c

[2] [3] [4] The 5 Most Popular Node.js Web Frameworks in 2025 - DEV Community
https://dev.to/leapcell/the-5-most-popular-nodejs-web-frameworks-in-2025-12po

[5] [6] [8] [9] [10] [11] SQL vs NoSQL: 5 Key Differences to Help You Choose
https://blog.skyvia.com/nosql-vs-sql/

[7] SQL vs NoSQL: Choosing the Right Database for An Application
https://blog.bytebytego.com/p/sql-vs-nosql-choosing-the-right-database

[12] [17] [18] [19] 12 FACTOR APP METHODOLOGY - DEV Community
https://dev.to/adeleke123/12-factor-app-methodology-1e57

[13] How can I transfer my game progress between different devices? – DECA Support
https://support.decagames.com/hc/en-us/articles/360035681412-How-can-I-transfer-my-game-progress-between-different-devices

[14] [15] Basic Tutorial for PouchDB and CouchDB · usePouchDB
https://terreii.github.io/use-pouchdb/docs/introduction/pouchdb_couchdb

[16] Replication - PouchDB
https://pouchdb.com/guides/replication.html